

National Institute of Technology, Rourkela
Mid Semester Examination, Autumn 2024
Compiler Design (CS3007)
Questions and answers

Duration: 2 Hours
Number of pages: 1

Full Marks: 30

- Attempt first question (Q1) and any four from the rest. In a grammar, upper case letters are non-terminals, rest symbols are terminals.

Q 1. Answer in one word or sentence.

[6 × 1]

- (a) Define compiler.
A system software that converts a source program in high-level language to a target program in assembly or machine-level language.
- (b) What is an assembler?
A system software which produces relocatable machine code for a machine dependent assembly program.
- (c) How an interpreter differs from a compiler?
An interpreter runs a program by converting (interpreting) each high-level statement of the program into machine code till the first error is encountered, whereas a compiler needs to generate and save the machine code for an entire error-free program before executing it.
- (d) Why do interpreted programs run slower than compiled programs?
An interpreter converts each high-level statement of the program into machine code leading slower execution of the program than a compiled program for which the machine code has been generated during the compilation process.
- (e) List the phases of compilation in proper order.
Macro/pre-processing, lexical analysis, syntactical analysis, semantic analysis, syntax-directed translation, intermediate code generation, machine-independent code optimization, register allocation and target code generation, machine-dependent code optimization.
- (f) Name an optional compilation phase.
Macro/pre-processing, code optimization.

Q 2. A single-line comment in a C program starts with (//) double forward slash. It extends till the end of the line and we don't need to specify its end. A multi-line comment starts with a forward slash star (/*) and ends with star forward slash (*/).

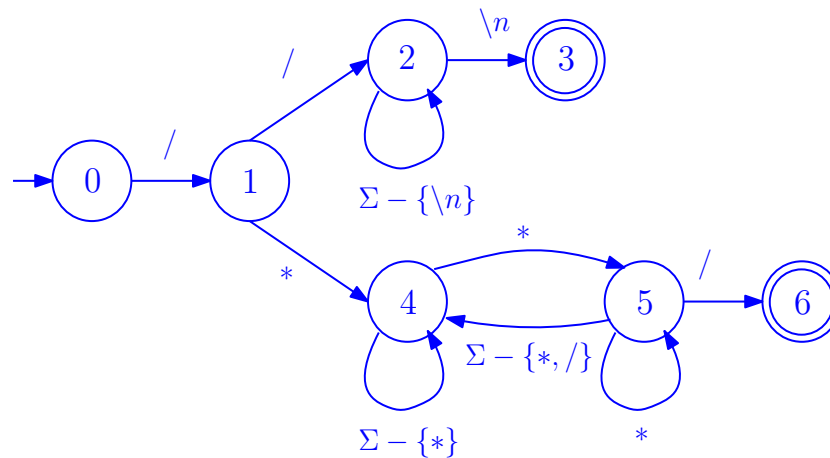
- (a) Write regular expressions for single and multi-line comments in C programming language. [2]
 C comments are the set of strings over alphabet $\Sigma = \{*, /, A, \diamond, \backslash n\}$ Here $\backslash n$ represents the new-line character, $\diamond$ represents any other whitespace character (like the space and tab characters), and A represents any non-whitespace character other than $*$ or $/$.

The regular expression for a single line comment is $//(\Sigma - \{\backslash n\})^*\backslash n$.

The regular expression for a multi-line comment is $/*(\Sigma^* - (*/)^*)*/$.

- (b) Construct a deterministic finite automata to recognize single and multi-line comments in C programs. [4]

The states 3 and 6 of the following deterministic finite automata accept single and multi-line comments in C programs, respectively.



Q 3. (a) Define the general form of a left-recursive grammar. Rewrite the same grammar after elimination of left-recursion. [2]

Let V_N and Σ be finite sets of non-terminals and terminals, respectively. A grammar G with left-recursion will have a rule of the form $A \rightarrow A\alpha_1|A\alpha_2|\dots|A\alpha_m|\beta_1|\beta_2|\dots|\beta_n$, where $A \in V_N$ and $\alpha_i, \beta_j \in (V_N \cup \Sigma)^*$, $\forall i \in \{1, 2, \dots, m\}$, $\forall j \in \{1, 2, \dots, n\}$, and β_j does not start with A .

Let G' be the version of G after elimination of left-recursion. G' will have left-recursion free rules of the form $A \rightarrow \beta_1 A'|\beta_2 A'|\dots|\beta_n A'$, $A' \rightarrow \alpha_1 A'|\alpha_2 A'|\dots|\alpha_m A'|\epsilon$, where $A, A' \in V_N$ and $\alpha_i, \beta_j \in (V_N \cup \Sigma)^*$, $\forall i \in \{1, 2, \dots, m\}$, $\forall j \in \{1, 2, \dots, n\}$.

(b) Consider the grammar G with rules $S \rightarrow aAc$, $A \rightarrow Ab|\epsilon$, where S is the starting symbol. Verify whether G is left-recursive and eliminate left-recursion in G to a new grammar G' . [4]

The rule $A \rightarrow Ab|\epsilon$ in G is left-recursive. Here $\alpha = b$ and $\beta = \epsilon$. Left-recursion is eliminated as $A \rightarrow \beta A' \implies A \rightarrow A'$, $A' \rightarrow \alpha A|\epsilon \implies A' \rightarrow bA'|\epsilon$. Thus, the left-recursion free version of G G' is $S \rightarrow aAc$, $A \rightarrow A'$, $A' \rightarrow bA'|\epsilon$.

Q 4. Consider the grammar G with rules $A \rightarrow Bb|Cd$, $B \rightarrow aB|\epsilon$, $C \rightarrow cC|\epsilon$, where A is the starting symbol. Find the FIRST and FOLLOW sets for non-terminals A, B and C . [6]

$$\begin{aligned} FIRST(A) &= FIRST(Bb) \cup FIRST(Cd) \\ &= FIRST(aB) \cup FIRST(b) \cup FIRST(cC) \cup FIRST(d) = \{a, b, c, d\}. \\ FIRST(B) &= FIRST(aB) \cup FIRST(\epsilon) = \{a, \epsilon\}. \\ FIRST(C) &= FIRST(cC) \cup FIRST(\epsilon) = \{c, \epsilon\}. \\ FOLLOW(A) &= \{\$ \}. FOLLOW(B) = \{b\}. FOLLOW(C) = \{d\}. \end{aligned}$$

Q 5. Consider the grammar G with rules $S \rightarrow cAd$, $A \rightarrow ab|a$, where S is the starting symbol.

- (a) Design a recursive descent parser using recursive procedures for the following grammar. The parser should be free of backtracking. [5]

Backtracking can be prevented if left-factoring is removed from the given grammar $S \rightarrow cAd$, $A \rightarrow aB$, $B \rightarrow b|\epsilon$. The backtracking free recursive-descent parser procedures are as follows.

S()
// $S \rightarrow cAd$

```
1. if *input_symbol_ptr = 'c' then
2.   input_symbol_ptr++;
3.   if A() then
4.     if *input_symbol_ptr = 'd' then
5.       input_symbol_ptr++;
6.       return true;
7.     endif
8.   endif
9. endif
10. return false;
```

A()
// $A \rightarrow aB$

```
1. if *input_symbol_ptr = 'a' then
2.   input_symbol_ptr++;
3.   if B() then
4.     return true;
5.   endif
6. endif
7. return false;
```

B()
// $B \rightarrow b|\epsilon$

```
1. if *input_symbol_ptr = 'b' then //  $B \rightarrow b$ 
2.   input_symbol_ptr++;
3.   return true;
4. else if *input_symbol_ptr = 'd' then //  $B \rightarrow \epsilon$ 
5.   return true;
6. endif
7. return false;
```

- (b) Give a trace for deriving the input string cad using the parser. [1]

Table 1: Derivation of cad

Stack	Derivation	Input	Action	Rule
\$	S	<u>cad</u> \$	Call S()	$S \rightarrow cAd$
$S()$	cAd	<u>cad</u> \$	c matched, input_symbol_ptr++	
$S()$	cAd	<u>cad</u> \$	Call A()	$A \rightarrow aB$
$S()A()$	cAd	<u>cad</u> \$	a matched, input_symbol_ptr++	
$S()A()$	cAd	<u>cad</u> \$	Call B()	
$S()A()B()$	$caBd$	<u>cad</u> \$	B() returns true	$B \rightarrow \epsilon$
$S()A()$	cad	<u>cad</u> \$	A() returns true	
$S()$	S	<u>cad</u> \$	d matched, input_symbol_ptr++	
$S()$	cad	<u>cad</u> \$	S() returns true	
\$	cad	<u>cad</u> \$	Accept	

Q 6. Consider the grammar G with rules $S \rightarrow cAd, A \rightarrow ab|a$, where S is the starting symbol.

- (a) Transform G to another grammar G' such that $G' \in LL(1)$. [1]

In G , for $A \rightarrow ab|a$ is a rule with left-factoring, $FIRST(ab) \cap FIRST(a) \neq \{\} \implies G \notin LL(1)$.

Left-factoring is removed to obtaining G' with rules $S \rightarrow cAd, A \rightarrow aB, B \rightarrow b|\epsilon$.

For G' , $FIRST(S) = \{c\}, FIRST(A) = \{a\}, FIRST(B) = \{b, \epsilon\}$.

$FOLLOW(S) = \{\$, \}, FOLLOW(A) = \{d\}, FOLLOW(B) = \{d\}$.

For $B \rightarrow b|\epsilon, FIRST(b) \cap FOLLOW(B) = \{\} \implies G' \in LL(1)$.

- (b) Construct the $LL(1)$ parsing table for G' . [4]

Table 2: $LL(1)$ parsing table for G'

	a	b	c	d	$\$$
S			$S \rightarrow cAd$		
A	$A \rightarrow aB$				
B		$B \rightarrow b$		$B \rightarrow \epsilon$	

- (c) Give a trace for deriving the input string cad using the parser. [1]

Table 3: Derivation of $cad\$$

Stack	Input	Action	Rule
$\$S$	$\underline{c}ad\$$	Pop S , push dAc	$S \rightarrow cAd$
$\$dAc$	$\underline{c}ad\$$	c matched, pop c , input_symbol_ptr++	
$\$dA$	$\underline{c}ad\$$	Pop A , push Ba	$A \rightarrow aB$
$\$dBa$	$\underline{c}ad\$$	a matched, pop a , input_symbol_ptr++	
$\$dB$	$\underline{c}ad\$$	Pop B	$B \rightarrow \epsilon$
$\$d$	$\underline{c}ad\$$	d matched, pop d , input_symbol_ptr++	
$\$$	$\underline{c}ad\$$	Accept	

Q 7. Given a grammar G with rules $A \rightarrow ab|b$. Verify whether

- (a) $G \in LL(1)$. [2]

$FIRST(ab) \cap FIRST(b) = \{a\} \cap \{b\} = \{\} \implies G \in LL(1)$.

- (b) $G \in LR(0)$. [2]

Augmented grammar with starting symbol S : $S \rightarrow A\$, A \rightarrow ab, A \rightarrow b$.

Set of items: $I_0 = CLOSURE(S \rightarrow \cdot A\$) = \{S \rightarrow \cdot A\$, A \rightarrow \cdot ab, A \rightarrow \cdot b\}$

$I_1 = GOTO(I_0, A) = CLOSURE(S \rightarrow A \cdot \$) = \{S \rightarrow A \cdot \$\}$

$I_2 = GOTO(I_0, a) = CLOSURE(A \rightarrow a \cdot b) = \{A \rightarrow a \cdot b\}$

$I_3 = GOTO(I_0, b) = CLOSURE(A \rightarrow b \cdot) = \{A \rightarrow b \cdot\}$

$I_4 = GOTO(I_1, \$) = CLOSURE(S \rightarrow A\$ \cdot) = \{S \rightarrow A\$ \cdot\}$

$I_5 = GOTO(I_2, b) = CLOSURE(A \rightarrow ab \cdot) = \{A \rightarrow ab \cdot\}$

All set of items are not having any shift-reduce conflicts. Hence $G \in LR(0)$.

- (c) $G \in SLR(1)$. [2]

Grammars belonging to the class $LR(0) \subset SLR(1)$ grammars. $G \in LR(0) \implies G \in SLR(1)$.

*****All the best*****